

Worst Case Scenario – The All Tails Scenario (The Flat List)

AND

The All Heads Scenario (The Traffic Jam)

1. The "All Tails" Scenario (The Flat List)

In this scenario, every single time a node was inserted, the `randomLevel()` coin flip immediately resulted in "Tails" (1). No node was ever promoted.

*The Header node is built to reach **Level 3**, but because no actual data nodes grew taller than **Level 0**, all the upper – level express lanes just point to empty space (NULL).*

```
[ SKIP LIST : ALL TAILS (MAX_LEVEL = 3) ]
```

```
Level 3: [Header] -----> NULL
Level 2: [Header] -----> NULL
Level 1: [Header] -----> NULL
Level 0: [Header] ---> [ 20 ] ---> [ 25 ] ---> [ 30 ] ---> NULL
```

Why it causes $O(n)$ time:

If we search for 30, the pointer drops instantly from Level 3 down to Level 0. Once on the ground floor, it has no choice but to physically step through 20, then 25, then 30. It operates exactly like a standard, singly linked list.

*SkipList * list*

Address *list*

ff33 *ff08*

Address *level* ** header*

ff08 **3** *ff12*

Address *key* ** forward*

ff12 -1 *ff80*

Address *Values*

<i>forward</i>	<i>Address</i>
<i>forward</i> [0]	<i>ff80</i>
<i>forward</i> [1]	<i>ff88</i>
<i>forward</i> [2]	<i>ff90</i>
<i>forward</i> [3]	<i>ff98</i>

<i>ff16</i>
<i>NULL</i>
<i>NULL</i>
<i>NULL</i>

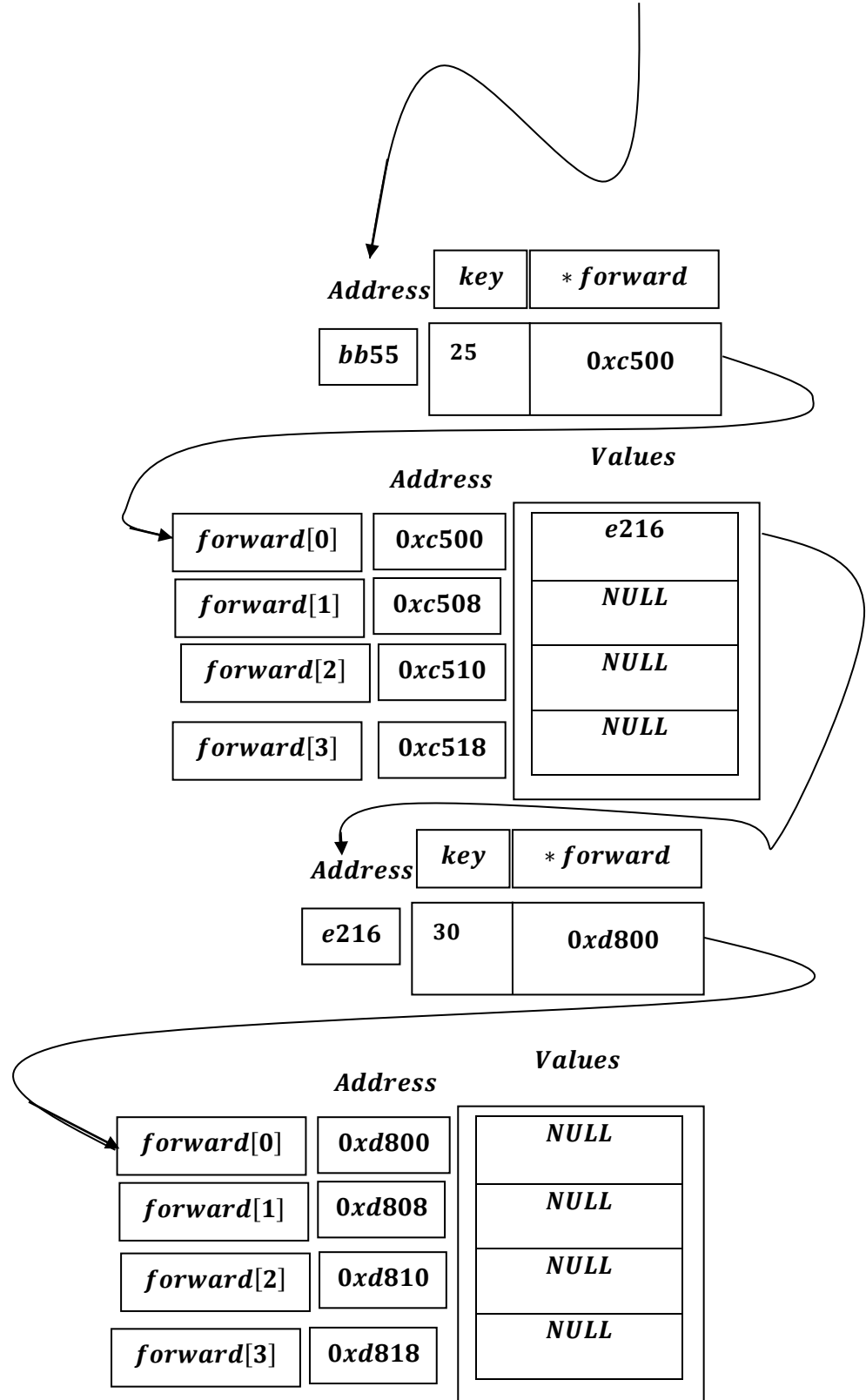
Address *key* ** forward*

ff16 20 *0xb400*

Address *Values*

<i>forward</i>	<i>Address</i>
<i>forward</i> [0]	<i>0xb400</i>
<i>forward</i> [1]	<i>0xb408</i>
<i>forward</i> [2]	<i>0xb410</i>
<i>forward</i> [3]	<i>0xb418</i>

<i>bb55</i>
<i>NULL</i>
<i>NULL</i>
<i>NULL</i>



The structure of a singly linked list.

2. The "All Heads" Scenario (The Traffic Jam)

In this scenario, the random number generator suffered a massive anomaly.

Every single time a node was inserted, it flipped Heads(0) repeatedly, growing to the absolute MAX_LEVEL cap.

Every single node exists on every single floor.

```
[ SKIP LIST : ALL HEADS (MAX_LEVEL = 3) ]
```

```
Level 3: [Header] ---> [ 20 ] ---> [ 25 ] ---> [ 30 ] ---> NULL
Level 2: [Header] ---> [ 20 ] ---> [ 25 ] ---> [ 30 ] ---> NULL
Level 1: [Header] ---> [ 20 ] ---> [ 25 ] ---> [ 30 ] ---> NULL
Level 0: [Header] ---> [ 20 ] ---> [ 25 ] ---> [ 30 ] ---> NULL
```

- **The Height of the Nodes:** The "All Heads" scenario as a situation where the random number generator consistently flips heads, "promoting every single node to the absolute maximum level (e.g., MAX_LEVEL = 3)".

And say, randomLevel or rLevel is also 3, in All Heads Scenario.

- **The Clogged Express Lane:** Because of this promotion, "every single node in the list has a pointer at level 3".
- **The Horizontal Traversal:** Because every node has a Level 3 pointer, the header's **forward[3]** pointer just points to Node 1. Node 1's **forward[3]** pointer points to Node 2. Node 2's **forward[3]** points to Node 3.

Similarly,

- **For Level 2 (*forward[2]*):** *Because every node has a Level 2 pointer, the header's *forward[2]* pointer just points to Node 1. Node 1's *forward[2]* pointer points to Node 2. Node 2's *forward[2]* points to Node 3.*
- **For Level 1 (*forward[1]*):** *Because every node has a Level 1 pointer, the header's *forward[1]* pointer just points to Node 1. Node 1's *forward[1]* pointer points to Node 2. Node 2's *forward[1]* points to Node 3.*
- **For Level 0 (*forward[0]*):** *Because every node has a Level 0 pointer, the header's *forward[0]* pointer just points to Node 1. Node 1's *forward[0]* pointer points to Node 2. Node 2's *forward[0]* points to Node 3.*

This is the ultimate proof of why the "All Heads" scenario is a structural disaster.

*SkipList * list*

Address *list*

ff33 *ff08*

Address *level* ** header*

ff08 **3** *ff12*

Address *key* ** forward*

ff12 -1 *ff80*

Address *Values*

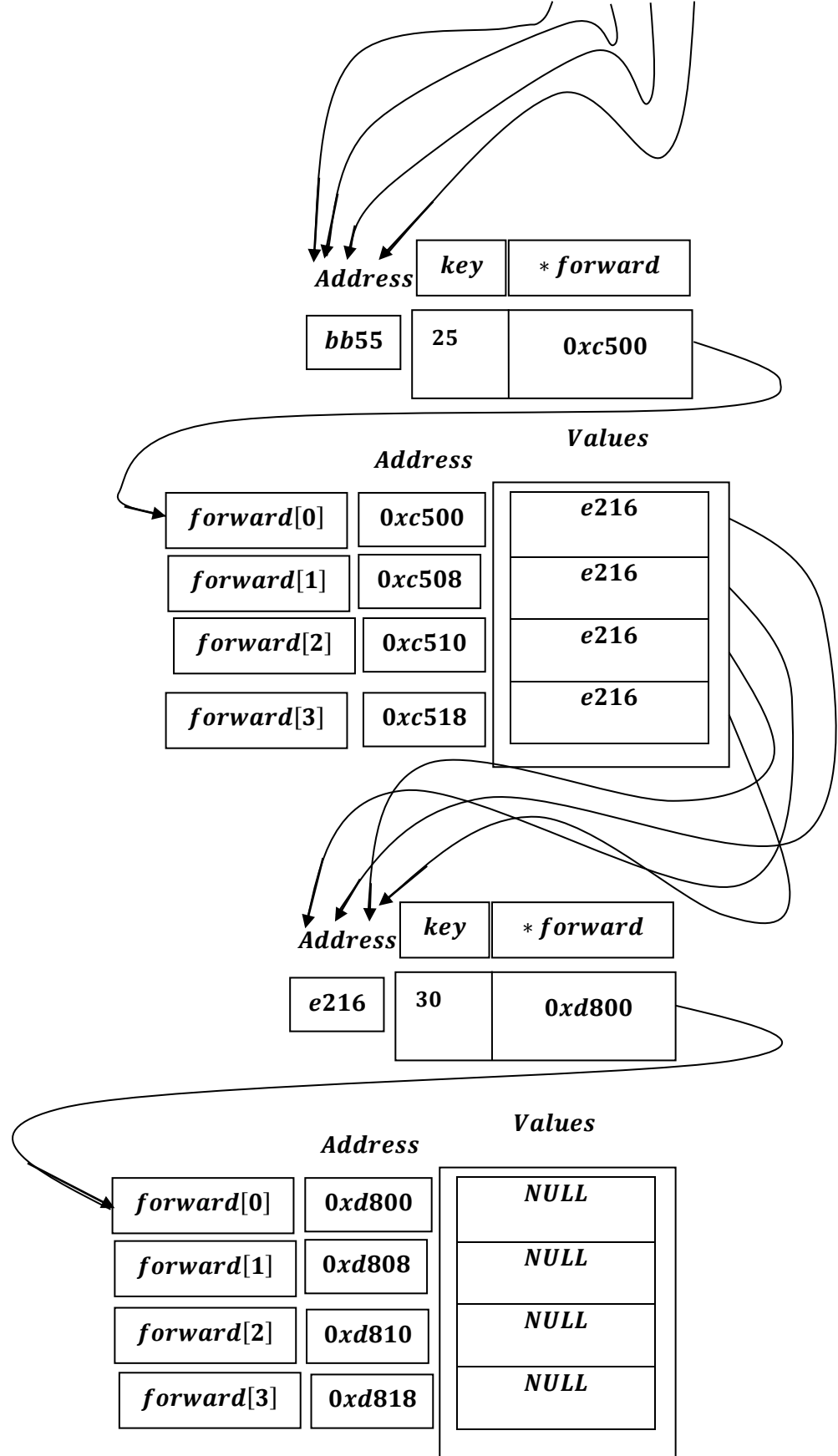
<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward</i> [0]	<i>ff80</i>	<i>ff16</i>
<i>forward</i> [1]	<i>ff88</i>	<i>ff16</i>
<i>forward</i> [2]	<i>ff90</i>	<i>ff16</i>
<i>forward</i> [3]	<i>ff98</i>	<i>ff16</i>

Address *key* ** forward*

ff16 20 *0xb400*

Address *Values*

<i>forward</i>	<i>Address</i>	<i>Values</i>
<i>forward</i> [0]	<i>0xb400</i>	<i>bb55</i>
<i>forward</i> [1]	<i>0xb408</i>	<i>bb55</i>
<i>forward</i> [2]	<i>0xb410</i>	<i>bb55</i>
<i>forward</i> [3]	<i>0xb410</i>	<i>bb55</i>



Why it causes $O(n)$ time:

Level 3 (The Traffic Jam): The algorithm starts at the top and is forced to evaluate 20, then 25, then 30 horizontally. It does a full $O(n)$ amount of work just crossing the top floor.

Dropping Down (Levels 2, 1, and 0): Once the pointer finally reaches Node 30 at the end of the list on Level 3, the for loop drops down to Level 2. Because the pointer is already sitting exactly where it needs to be, the lower – level horizontal loops terminate instantly with zero steps taken. This repeats for Level 1 and Level 0.

Level 2, Level 1, and Level 0 are completely skipped horizontally because all the heavy lifting was already accidentally done on the top floor.

- **Top Floor (Level 3):** The pointer starts at the Header, moves right to **Node 1 (20)**, moves right to **Node 2 (25)**, and finally moves right to **Node 3 (30)**. It has now reached the end of the list on **Level 3**. The **while** loop terminates.
- **Dropping to Level 2:** The outer **for** loop executes **$i--$** , dropping the floor down to **Level 2**.
 - **The Catch:** The **current** pointer does not reset! It is already physically sitting on Node 30.
 - When the inner while loop asks, Should I move right?, it checks Node 30's **forward[2]** pointer. Because **Node 30** is the last node, that pointer is **NULL**.
 - The condition fails instantly, and zero horizontal steps are taken.

- **Dropping to Level 1 and Level 0:** The exact same thing happens.

The floor drops, but because the pointer is already parked

at the very end of the list(NULL), the while loop condition instantly fails on every lower floor.

How?

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

*Here is exactly how that code executes as the **for** loop drops down:*

1. **Level 2 (i = 2):** The program checks the condition **current → forward[2] ≠ NULL**. Because Node 30 is the last node, **current → forward[2]** is NULL. The condition fails instantly, the **while** loop terminates with zero steps taken, and the program simply records **update[2] = current** (the pointer address of Level3 where Node 30 is sitting, here **e216**).
2. **Level 1 (i = 1):** The for loop drops down, but current is still Node 30. It checks **current → forward[1] ≠ NULL**. But **current → forward[1]** is NULL. The condition fails instantly, the while loop is skipped, and it records **update[1] = current** (the pointer address of Level3 where Node 30 is sitting, here **e216**).
3. **Level 0 (i = 0):** The exact same thing happens. It checks **current → forward[0] ≠ NULL**. It is NULL. The condition fails instantly, the while loop is skipped, and it records **update[0] = current** (Node 30).

Insert Element – When All Heads Scenario

Now Come to Insert Element:

```
current = current->forward[0];
```

current = current → forward[0] = NULL.

```
int rlevel = randomLevel();
```

rlevel = randomLevel() = 3[MAX_LEVEL]

*Runs Theoretically $O(\log n)$ times
But in practical scenario, it is
fixed constant = $O(1)$ as it cannot
go above MAX_LEVEL which is a fixed
constant.*

```
if (rlevel > lst->level) {  
    for (int i = lst->level + 1; i <= rlevel; i++) {  
        update[i] = lst->header;  
    }  
    lst->level = rlevel;  
}
```

Here rlevel = 3 > lst → level = 3, completely skipped.

```
struct Node *n = createNode(key, rlevel);  
for (int i = 0; i <= rlevel; i++) {  
    n->forward[i] = update[i]->forward[i];  
    update[i]->forward[i] = n;  
}  
cout << "Successfully inserted key " << key << endl;  
}
```

→ Now, it will create a new Node.

for $i = 0; i \leq rlevel() = 3$:

$n \rightarrow forward[0] = update[0]$ i.e. e216 → forward[0] = NULL

update[0] i.e. e216 → forward[0] = n i.e. New Node's Address

$i = 0 + 1 = 1$

for i = 1; i ≤ rlevel() = 3:

n → forward[1] = update[1] i. e. e216 → forward[1] = NULL

update[1] i. e. e216 → forward[1] = n i. e. New Node's Address

i = 1 + 1 = 2

for i = 2; i ≤ rlevel() = 3:

n → forward[2] = update[2] i. e. e216 → forward[2] = NULL

update[2] i. e. e216 → forward[2] = n i. e. New Node's Address

i = 2 + 1 = 3

for i = 3; i ≤ rlevel() = 3:

n → forward[3] = update[3] i. e. e216 → forward[3] = NULL

update[3] i. e. e216 → forward[3] = n i. e. New Node's Address

i = 3 + 1 = 4

for i = 4; i ≤ rlevel() = 4 [Loop Exits]

Hence , new Node is adjusted at very end of the list.

In practical Scenario: MAX_LEVEL = RLEVEL = O(1)

as MAX_LEVEL is constant.

Therefore , for loop ran from 0 to some constant `c` resulting O(1).

In Theoretical Scenario: MAX_LEVEL = RLEVEL = O(log n)

Therefore , for loop ran from 0 to log n resulting O(log n).

At both the scenarios: $T_{Search} + T_{coinflip} + T_{insert\&Rewire} =$

= Search: $O(n)$ + CoinFlip: $O(\log n)$ or $O(1)$ + Insert: $O(\log n)$ or $O(1)$ = $O(n)$

Delete Element – When All Heads Scenario

```
current = current->forward[0];
```

current = current → forward[0] = NULL.

```
if (current != NULL && current->key == key) {
    // Rewire the pointers to bypass the deleted node
    for (int i = 0; i <= lst->level; i++) {
        if (update[i]->forward[i] != current) {
            break; // If a level doesn't point to current, higher levels
won't
            // either
        }
        update[i]->forward[i] = current->forward[i];
    }

    // Free memory using free()
    free(current->forward);
    free(current);

    // Reduce the maximum level of the list if layers become empty
    while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
        lst->level--;
    }
    cout << "Successfully deleted key " << key << endl;
}
```

As Current is NULL ,

- *whole rewiring is skipped.*
- *The freeing will get skipped.*
- *while loop i.e.Reducing Maximum Level also gets skipped*

```
else {
    cout << "Key " << key << " not found. Cannot delete." << endl;
}
}
```

And we run: ``Key is not found``, hence:

Hence : Search: $O(n) + O(1) = O(n)$

Same thing for All Tails Scenario:

Search:

As, at first current will be pointing to Header Node : ff12 and

as , ff12 $\rightarrow$ forward[3] = NULL , ff12 $\rightarrow$ forward[2] = NULL ,

ff12 $\rightarrow$ forward[1] = NULL.

Therefore:

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

update[1] = ff12

update[2] = ff12

update[3] = ff12

When at Level = 0 :

current $\rightarrow$ e216

update[0] = e216

After Search , Time Complexity = $O(n)$

Insert Element – When All Tails Scenario

```
current = current->forward[0];
```

Current = Current → forward[0] = NULL

```
if (current == NULL || current->key != key) {
    int rlevel = randomLevel();

    // If the random level is greater than the list's current max
    level,
    // initialize the update array for the new levels to point to
    the header.
    if (rlevel > lst->level) {
        for (int i = lst->level + 1; i <= rlevel; i++) {
            update[i] = lst->header;
        }
        lst->level = rlevel;
    }
}
```

As Current = NULL(True):

rlevel = randomLevel(), Tails therefore return 0. $\left[\begin{array}{l} \text{At first} \\ \text{coin flip,} \\ \text{hence strict} \\ \text{O(1).} \end{array} \right]$

if(rlevel = 0 > lst → level = 3)[True]:

for $\left(i = \text{lst} \rightarrow \text{level} + 1[3 + 1 = 4]; i \leq \text{rlevel} = 0 \left[\begin{array}{l} \text{Loop} \\ \text{Condition} \\ \text{Fails} \\ \text{Hence Exit} \end{array} \right] \right)$

lst → level = rlevel = 0.

```

struct Node *n = createNode(key, rlevel);
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
cout << "Successfully inserted key " << key << endl;
}

```

→ Now , it will create a new Node.

for i = 0; i ≤ rlevel() = 0:

n → forward[0] = update[0] i. e. e216 → forward[0] = NULL

update[0] i. e. e216 → forward[0] = n i. e. New Node's Address

i = 0 + 1 = 1

for i = 0; i ≤ rlevel() = 0 [Loop Exits]

Hence , new Node is adjusted at very end of the list.

*As , loop strictly runs only Once , hence Time Complexity of Insertion and Rewiring is strictly constant: **O(1)**.*

$$\begin{aligned}
 &\therefore T_{Search} + T_{coinflip} + T_{insert\&Rewire} = \\
 &= \text{Search: } O(n) + \text{CoinFlip: } O(1) + \text{Insert: } O(1) = O(n)
 \end{aligned}$$

Delete Element – When All Tails Scenario

```
current = current->forward[0];
```

current = current → forward[0] = NULL.

```
if (current != NULL && current->key == key) {
    // Rewire the pointers to bypass the deleted node
    for (int i = 0; i <= lst->level; i++) {
        if (update[i]->forward[i] != current) {
            break; // If a level doesn't point to current, higher levels
won't
            // either
        }
        update[i]->forward[i] = current->forward[i];
    }

    // Free memory using free()
    free(current->forward);
    free(current);

    // Reduce the maximum level of the list if layers become empty
    while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
        lst->level--;
    }
    cout << "Successfully deleted key " << key << endl;
}
```

As Current is NULL ,

- *whole rewiring is skipped.*
- *The freeing will get skipped.*
- *while loop i.e.Reducing Maximum Level also gets skipped*

```
else {
    cout << "Key " << key << " not found. Cannot delete." << endl;
}
}
```

And we run: ``Key is not found``, hence:

Hence : Search: $O(n) + O(1) = O(n)$
